Data Management

2023/2024

Graph Databases

Applications in Neo4J

SAPIENZA
UNIVERSITÀ DI ROMA

Tutor
Roberto Maria Delfino

delfino@diag.uniroma1.it

Lecture contents

- Motivations

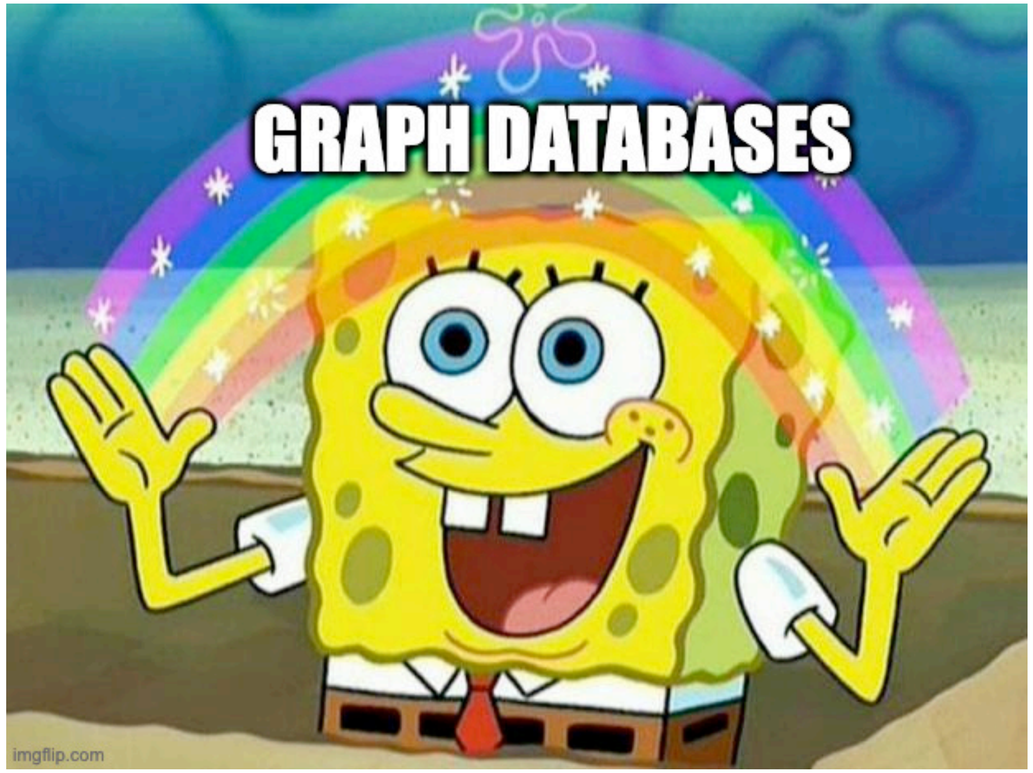
- Case study: recommendation system

- Advantages of graph representation

- RDBM vs GDBMS: pros & cons

- Neo4J
  - Overview
  - The property graph model
  - The Cypher query language
  - CRUD operations in Cypher

- Demo



A new type of data

What is the data generated by these companies characterised by?

Focus on relationships

- Social Networks
- Online matching
- Content sharing
- Online markets
- Contact tracing
- others

The Relational Model

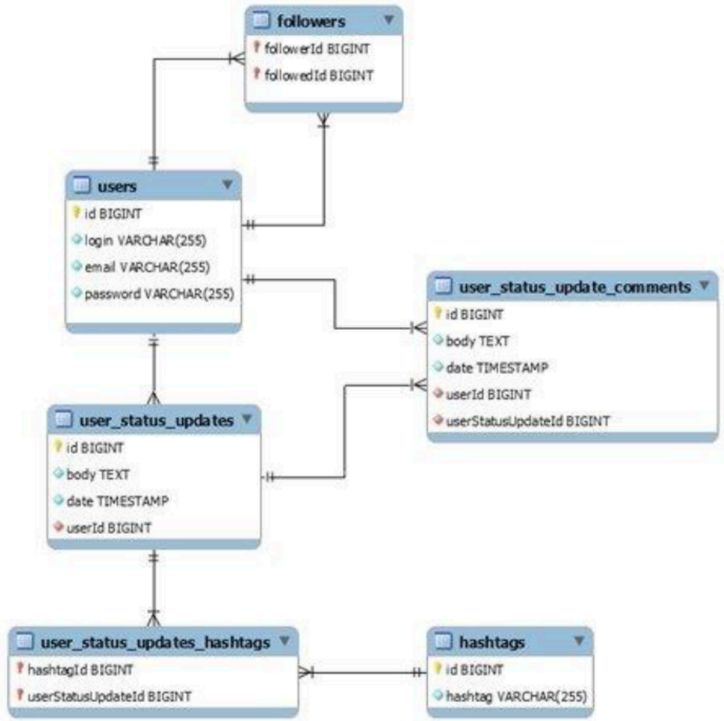
Despite the name, the relational model does **not** allow for explicit representation of relationships among data, which have to be represented through JOIN operations.

Drawback: JOIN operations are the bottleneck of query answering in relational DBs

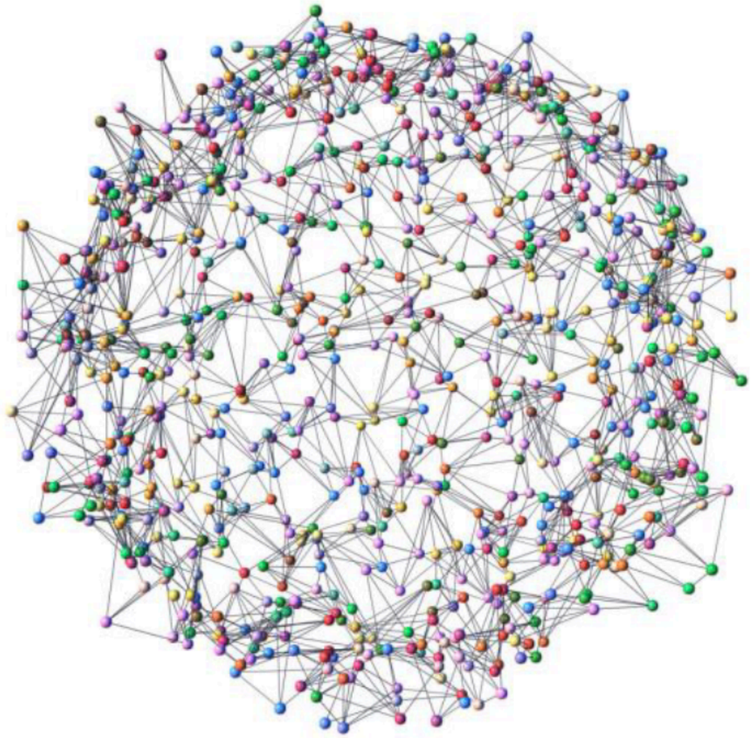
Solution: represent relationships among data as explicit data

Data structures inherently based on the idea of relationships: **Graphs**

A different perspective



Relationships = Joins



Relationships = Edges

Case study: recommendation system

User

UID	Name
A10	Alice
B13	Bob
…	…
Z71	Zach

Product

PID	Desc
P07X	Diapers
P31Y	Beer
…	…
P57Z	Shoes

Review

Usr	Prod	Rating
B13	P57Z	4
A10	P31Y	1
…	…	…
B13	P31Y	3

Case study: recommendation system

User

UID	Name
A10	Alice
B13	Bob
…	…
Z71	Zach

Product

PID	Desc
P07X	Diapers
P31Y	Beer
…	…
P57Z	Shoes

Review

Usr	Prod	Rating
B13	P57Z	4
A10	P31Y	1
…	…	…
B13	P31Y	3

Goal: given a user u, find identifiers and descriptions of all products which u could be interested in.

How?

Case study: recommendation system

User

UID	Name
A10	Alice
B13	Bob
…	…
Z71	Zach

Product

PID	Desc
P07X	Diapers
P31Y	Beer
…	…
P57Z	Shoes

Review

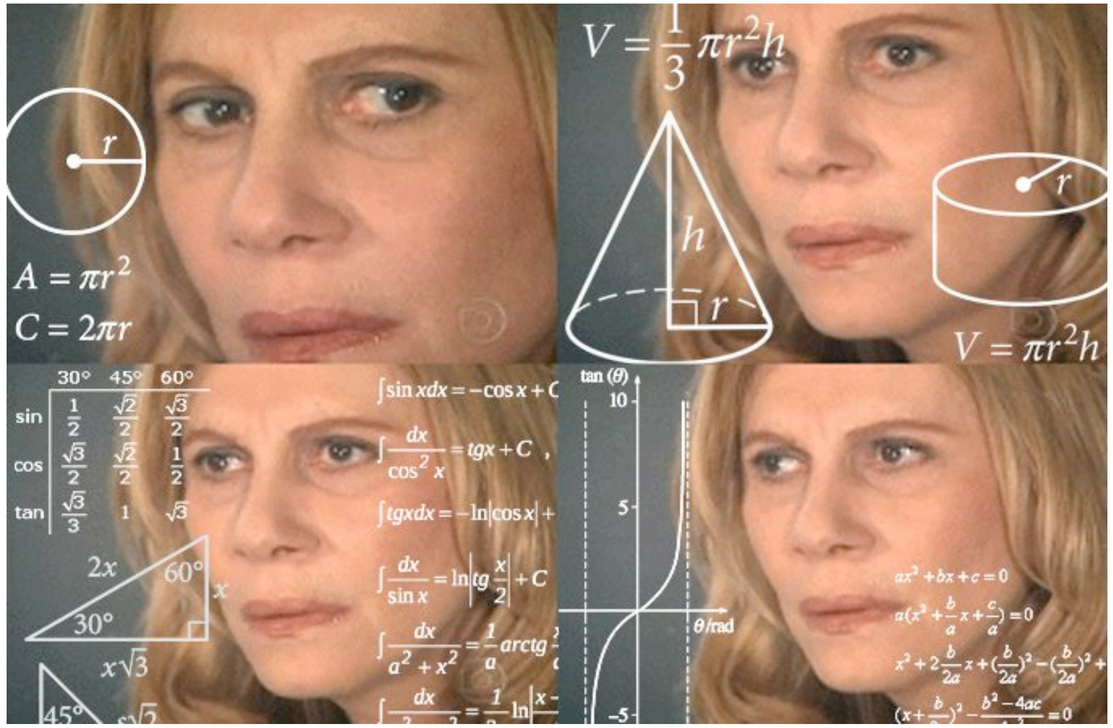
Usr	Prod	Rating
B13	P57Z	4
A10	P31Y	1
…	…	…
B13	P31Y	3

Goal: given a user u, find identifiers and descriptions of all products which u could be interested in.

How: find similar users (e.g., those who gave a high evaluation to the same products u did) and their other high rated products, and suggest them to user u.

SQL query: any idea?

SQL query: any idea?



Not easy at all!

A possible solution in SQL

```
SELECT DISTINCT Product.PID, Product.Desc

FROM Product, User u

JOIN Review r1 ON u.UID = r1.Usr

JOIN Review r2 ON r1.Prod = r2.Prod

JOIN Review r3 ON r2.Usr = r3.Usr AND (r3.rating = 4 OR r3.rating = 5) AND r3.Prod

NOT IN (

  SELECT Prod

  FROM Review JOIN User ON Review.Usr = User.UID

  WHERE Usr = "Alice")

WHERE u.name = "alice" AND r1.Usr = 'a' AND (r1.rating = 4 OR r1.rating = 5) AND (r2.rating = 4 OR r2.rating
= 5) AND r1.Usr != r2.Usr AND Product.PID = r3.Prod;
```

The cost in a RDBMS (unfair version)

Query | Query History

```
1  SELECT DISTINCT r3.asin
2  FROM public."user" u
3  JOIN public."review" r1 ON u.uid = r1.uid
4  JOIN public."review" r2 ON r1.asin = r2.asin
5  JOIN public."review" r3 ON r2.uid = r3.uid AND (r3.rating = 4 OR r3.rating = 5) AND r3.asin
6  NOT IN (
7    SELECT asin
8    FROM public."review" JOIN public."user" ON public."review".uid = public."user".uid
9    WHERE public."user".uid = 'AQ4B80F0JSBNI')
10 WHERE r1.uid = 'AQ4B80F0JSBNI' AND (r1.rating = 4 OR r1.rating = 5)
11 AND (r2.rating = 4 OR r2.rating = 5) AND r1.uid != r2.uid
```

Scratch Pad ×

Data Output | Messages | Explain × | Notifications

	asin character (100)
1	555751659X
2	555820690X
3	7799420340
4	9434682614
5	9714721180
6	9721717150
7	B0000025WT
8	B000006045
9	B000006EHH
10	B000006Z18
11	B00000747R
12	B0000074HI
13	B000007UQY
14	B000007WCV

Total rows: 1000 of 3905 | Query complete 01:15:06.834 | Ln 2, Col 22

Elapsed time to execute the query:

1 h 15 m 6 s

Amazon Web Reviews Dataset
(digital music sub-dataset)

https://snap.stanford.edu/data/web-Amazon.html

The cost in a RDBMS (fair version)

Query | Query History

```
1  SELECT DISTINCT r3.asin
2  FROM public."user" u
3  JOIN public."review" r1 ON u.uid = r1.uid
4  JOIN public."review" r2 ON r1.asin = r2.asin
5  JOIN public."review" r3 ON r2.uid = r3.uid AND (r3.rating = 4 OR r3.rating = 5) AND r3.asin
6  NOT IN (
7    SELECT asin
8    FROM public."review" JOIN public."user" ON public."review".uid = public."user".uid
9    WHERE public."user".uid = 'AQ4B80F0JSBNI')
10 WHERE r1.uid = 'AQ4B80F0JSBNI' AND (r1.rating = 4 OR r1.rating = 5)
11 AND (r2.rating = 4 OR r2.rating = 5) AND r1.uid != r2.uid
12
```

Scratch Pad ×

Data Output | Messages | Notifications

	asin character (100)
1	555751659X
2	555820690X
3	7799420340
4	9434682614
5	9714721180
6	9721717150
7	B0000025WT
8	B000006045
9	B000006EHH
10	B000006718

Total rows: 1000 of 3905 | Query complete 00:02:29.337 | Ln 1, Col 24

By using indexes, the elapsed time dramatically drops to:

2 m 29 s

Amazon Web Reviews Dataset
(digital music sub-dataset)

https://snap.stanford.edu/data/web-Amazon.html

The cost in a GDBMS

```
1  //User_recommendations
2  MATCH (c:Customer)-[r1]→(p:Product)←[r2]-(sc:Customer)-[r3]→(sp:Product)
3  WHERE c.uid ='AQ4B80F0JSBNI'
4  AND (r1.rating = '4.0' OR r1.rating = '5.0')
5  AND (r2.rating = '4.0' OR r2.rating = '5.0')
6  AND (r3.rating = '4.0' OR r3.rating = '5.0')
7  AND NOT (c)-[]→(sp)
8  RETURN DISTINCT sp.asin
```

Table | Text | Code

	sp.asin
1	"B00U2RMDQK"
2	"B00T0ZHMGG"
3	"B00QHDM7HS"
4	"B004136V1O"
5	"B0040HNAG0"
6	"B000HCSJTK"
7	

Started streaming 3905 records after 2 ms and completed after 67 ms, displaying first 1000 rows.

The equivalent query executed over a Graph DBMS through graph navigation takes:

67 milliseconds

Amazon Web Reviews Dataset
(digital music sub-dataset)

https://snap.stanford.edu/data/web-Amazon.html

When to use a Graph Database

- Data is **highly-connected**: otherwise you would come up with a **sparsely** connected graph (e.g., if you are interested in information about people but you don’t care about their relationships, then a relational database or a document-based DBMS is better)

- **Retrieving** data is more important than storing it (if writing is more important than carrying out complex queries and complex analyses, then stick with a relational database)

- Data model **changes frequently** (relational databases require the **schema** to be defined and designed in advance and are not suitable for frequent changes, while a graph does not rely on a rigid predefined schema)

When not to use a Graph Database

- Queries do not include a specific **starting point**. Graph databases fit well when you have to traverse them from specific starting points, while they are not suitable for frequent scans of the whole graph

- You need **key/value** storage. When you have a known key and need to retrieve all data associated with it, graph databases may not be the best choice (e.g., storing personal information and retrieving it by name or ID - on the contrary, if there were other entities involved (e.g., visited locations) and several connections are required to map users, then a graph database would do the job)

- You need to store large **chunks** of information. If entities in your model have very large attributes (BLOBs, CLOBs, long texts) then graph databases are not the best choice

Relational model vs graph model

There is no one-fits-all data representation model. Each model comes with advantages and disadvantages.

	Relational	**Graph**
Query Language	SQL	No standard exists
Relationships-heavy	Join operations are costly	Graph traversal is cheap
Schema	Defined a priori	No schema, prone to inconsistencies
Data heterogeneity	Requires Data Integration	Elastic wrt heterogeneity
User base	Huge	Far smaller
Data change frequently	Limited by schema	Changes can be applied with ease

Other advantages: (Spectral) Graph Theory

Properties of graphs can be easily analysed by means of the **spectra** of matrices associated with them (e.g., the set of their eigenvalues)

Some common problems:

- **Shortest path** between nodes (powers of the adjacency matrix)
  Use-cases: navigation systems, TLC networking, social network analysis

- **Community detection** (smallest eigenvalues of the Laplacian matrix)
  Use-cases: recommendation systems, biological networks

- **Centrality** (spectrum of the adjacency matrix)
  Use cases: urban planning, power grid analysis, PageRank

DBMS Market Today

Rank			DBMS	Database Model	Score		
Mar 2024	Feb 2024	Mar 2023			Mar 2024	Feb 2024	Mar 2023
1.	1.	1.	Oracle	Relational, Multi-model	1221.06	-20.39	-40.23
2.	2.	2.	MySQL	Relational, Multi-model	1101.50	-5.17	-81.29
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	845.81	-7.76	-76.20
4.	4.	4.	PostgreSQL	Relational, Multi-model	634.91	+5.50	+21.08
5.	5.	5.	MongoDB	Document, Multi-model	424.53	+4.18	-34.25
6.	6.	6.	Redis	Key-value, Multi-model	157.00	-3.71	-15.45
7.	7.	↑ 8.	Elasticsearch	Search engine, Multi-model	134.79	-0.95	-4.28
8.	8.	↓ 7.	IBM Db2	Relational, Multi-model	127.75	-4.47	-15.17
9.	9.	↑ 11.	Snowflake	Relational	125.38	-2.07	+10.98
10.	10.	↓ 9.	SQLite	Relational	118.16	+0.88	-15.66
11.	11.	↓ 10.	Microsoft Access	Relational	107.93	-5.24	-24.13
12.	12.	12.	Cassandra	Wide column, Multi-model	104.59	-4.69	-9.20
13.	13.	13.	MariaDB	Relational, Multi-model	95.03	-2.20	-1.81
14.	14.	14.	Splunk	Search engine	89.68	-1.97	+1.71
15.	↑ 16.	↑ 16.	Microsoft Azure SQL Database	Relational, Multi-model	78.51	-1.06	+1.06
16.	↓ 15.	↓ 15.	Amazon DynamoDB	Multi-model	77.72	-5.18	-3.05
17.	17.	↑ 19.	Databricks	Multi-model	74.34	-2.57	+13.48
18.	18.	↓ 17.	Hive	Relational	64.82	-0.99	-6.09
19.	19.	↑ 21.	Google BigQuery	Relational	62.67	-0.96	+9.23
20.	20.	↓ 18.	Teradata	Relational, Multi-model	48.95	-2.29	-14.79
21.	21.	↑ 22.	FileMaker	Relational	48.81	-1.67	-2.33
22.	↑ 23.	↑ 23.	SAP HANA	Relational, Multi-model	45.49	+0.27	-5.36
23.	↓ 22.	↓ 20.	Neo4j	Graph	44.36	-2.25	-9.15

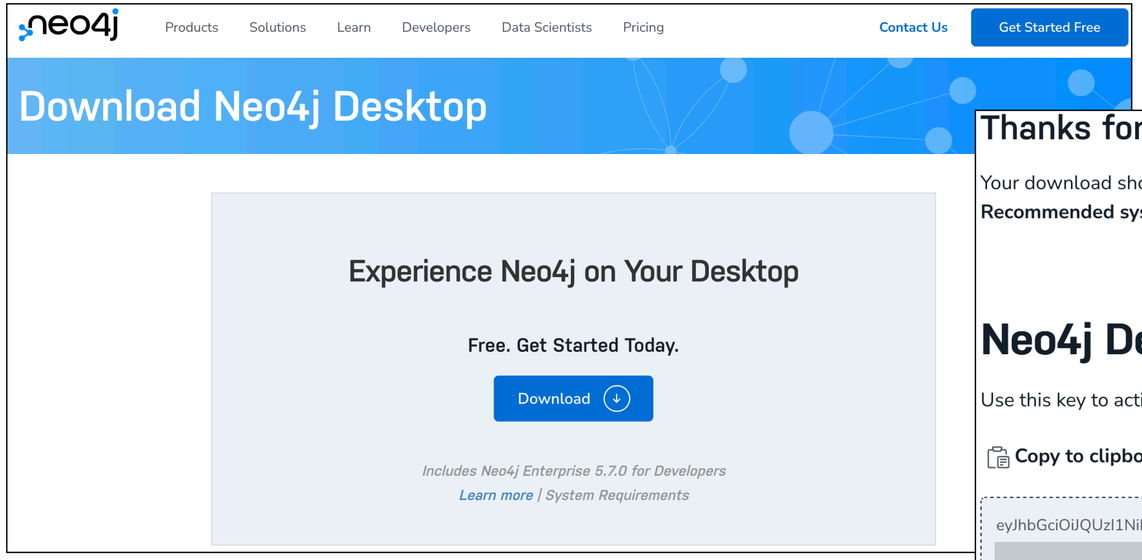
source: db-engines.com/en/ranking

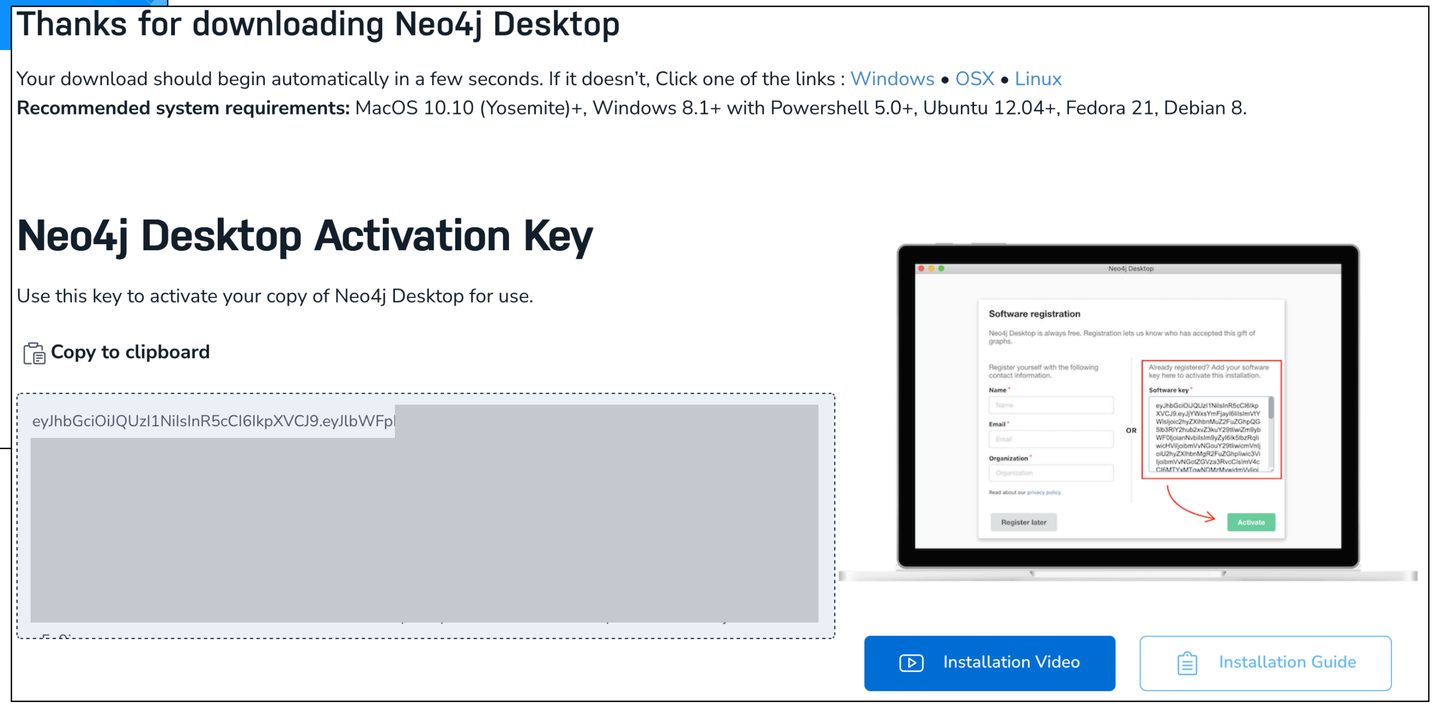
Graph Database Management Systems

Graph	Multi-model
Neo4J	Microsoft Azure Cosmos DB
JanusGraph	Virtuoso
NebulaGraph	ArangoDB
Memgraph	OrientDB
TigerGraph	Amazon Neptune
Dgraph	GraphDB
Apache Giraph	AllegroGraph
…	…

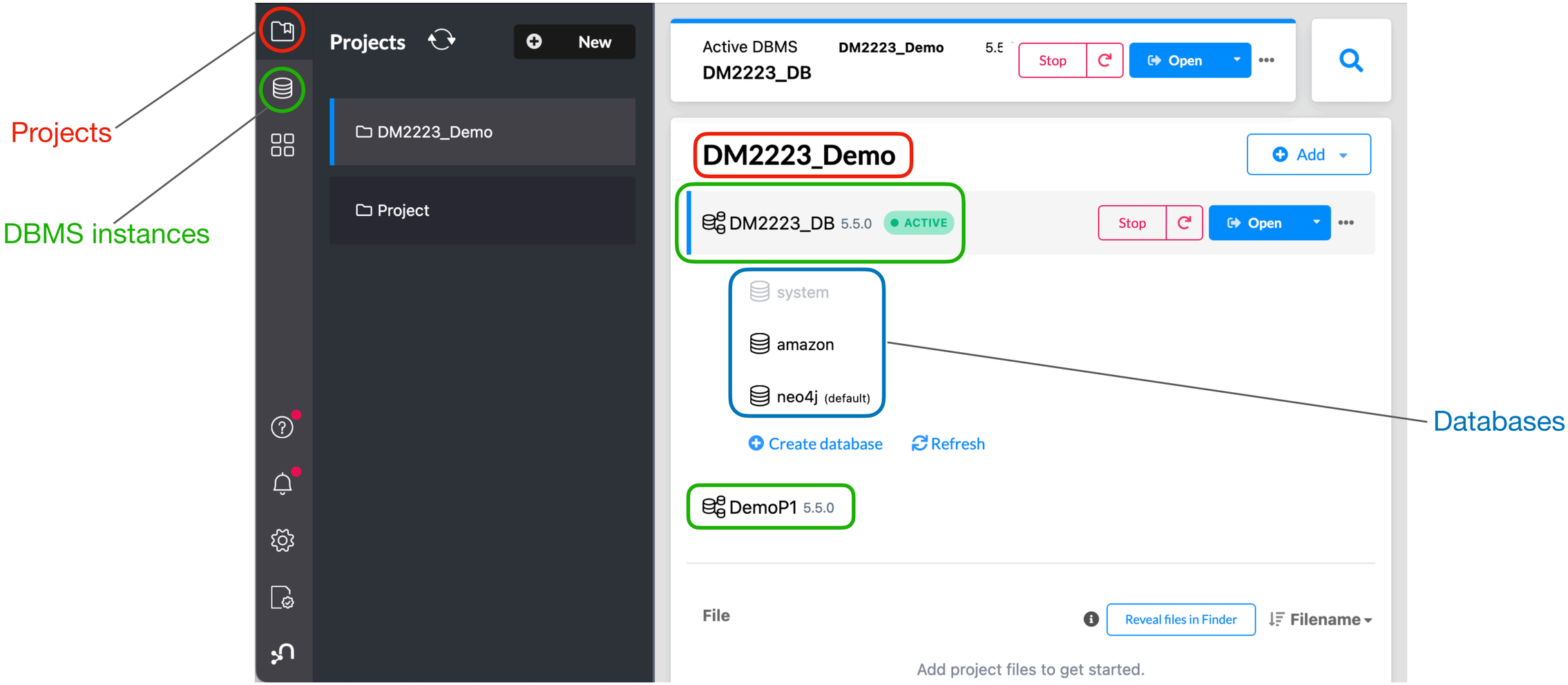
Neo4J Desktop

https://neo4j.com/download/

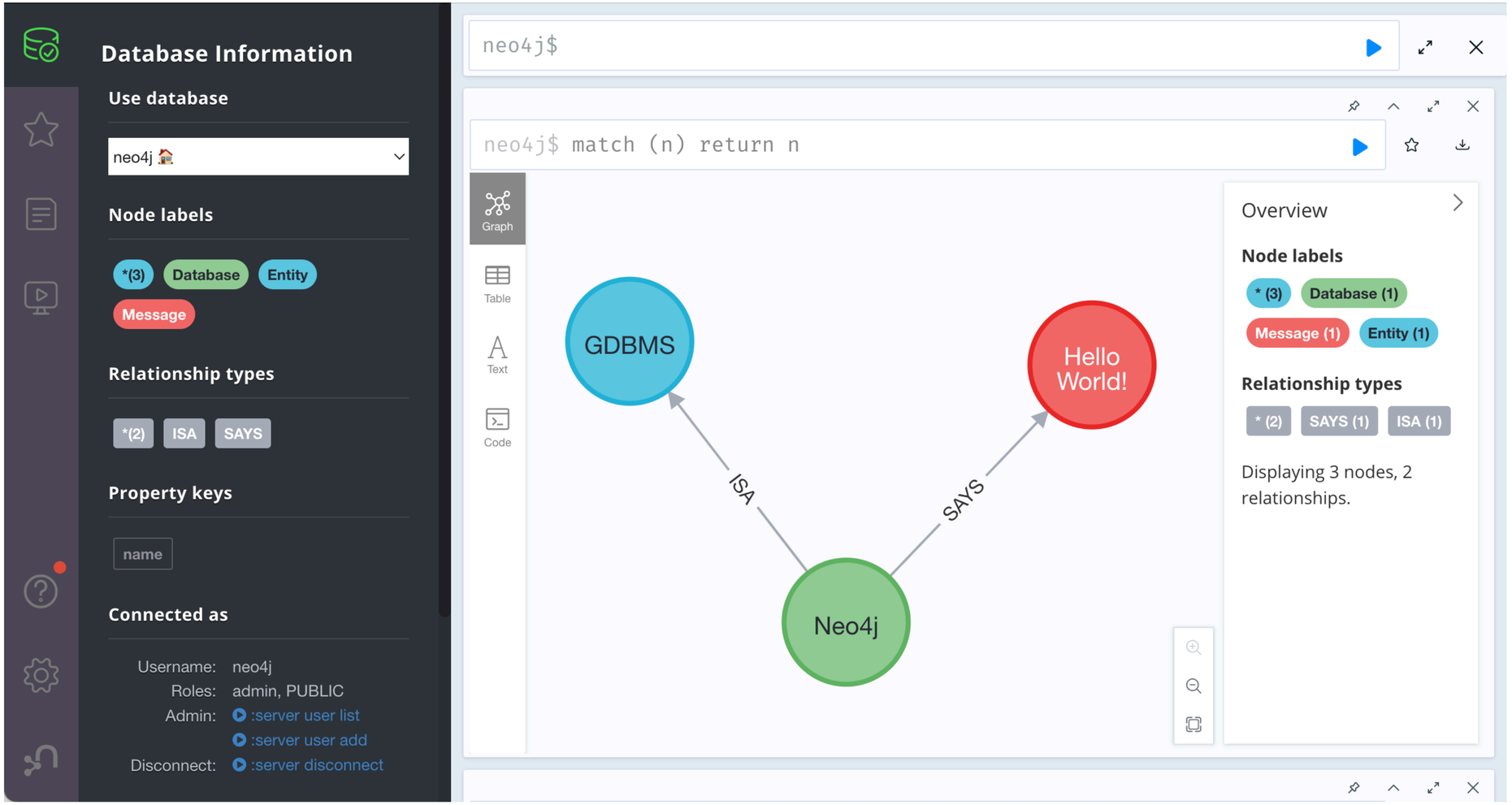




Neo4J Desktop

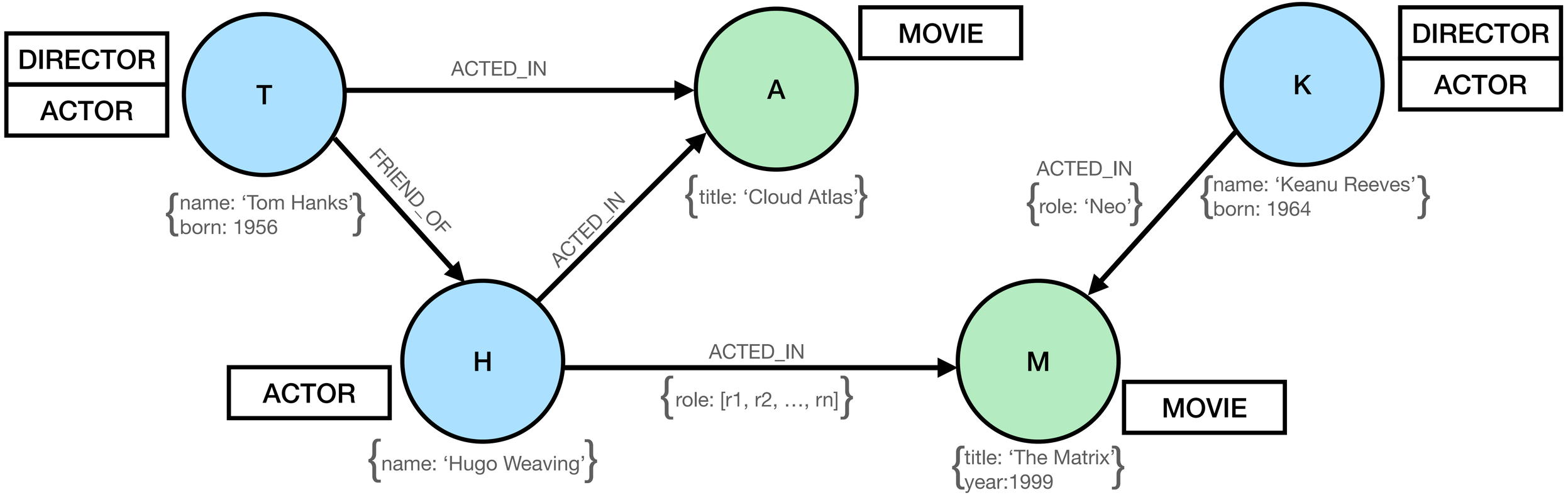


Neo4J Desktop - Browser



Neo4J - Property graphs

Keywords: nodes, edges, labels, properties, patterns



General rules of the Neo4J property graph

Node:
- can have Relationships
- can have Properties
- can have Labels

Relationship:
- must have a both a starting and an ending Node
- must have a Type (similar to a Label, uniquely identified by its name)
- can have Properties

Property:
- must have a key and a(n array of) value(s)

Label:
- must have a name
- groups Nodes sharing it as an entity type

Neo4J’s query language: Cypher

Declarative SQL-inspired query language based on **pattern-matching**

Support for CRUD operations:

- **C** - Create nodes and/or links
- **R** - Read portions of the graph matching specific patterns
- **U** - Update nodes and/or links
- **D** - Delete nodes and/or link

Neo4J’s query language: Cypher

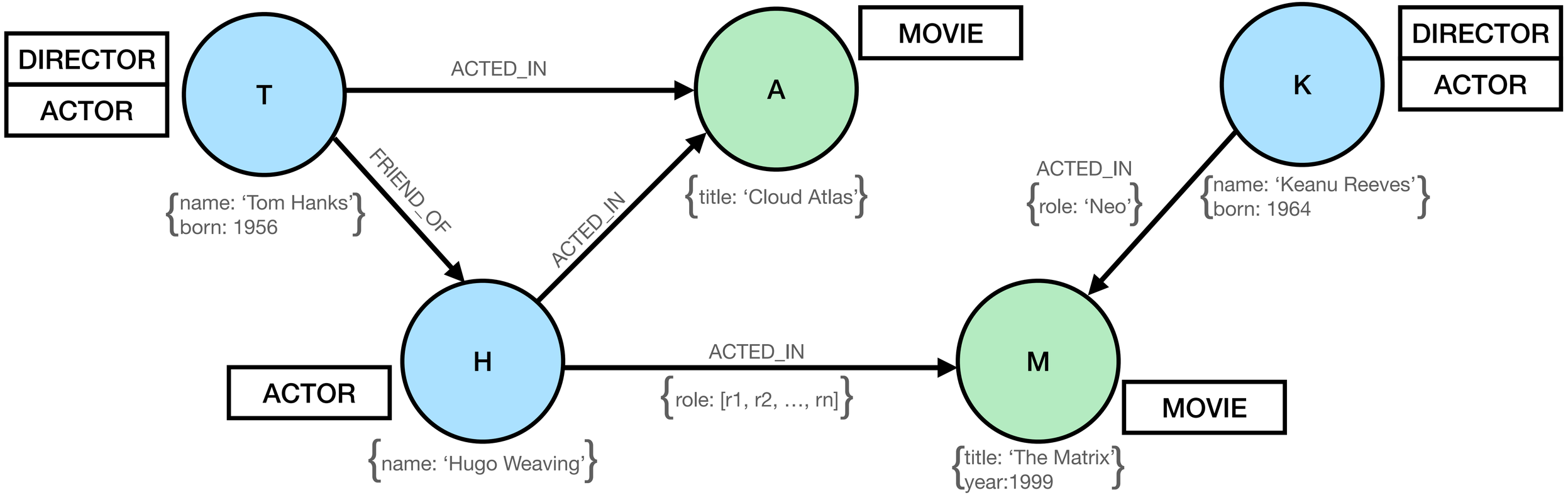
Cypher allows you to match specific portions of a graph through a visual ASCII-Art syntax

- Nodes are incapsulated in rounded brackets: **(node)**
- Relationships are represented through arrows: **-[:EDGE]->**
- Properties are encapsulated in curly brackets: **{p1:v1, p2:[v2,v3,…]}**
- Labels follow the colon symbol

Example:

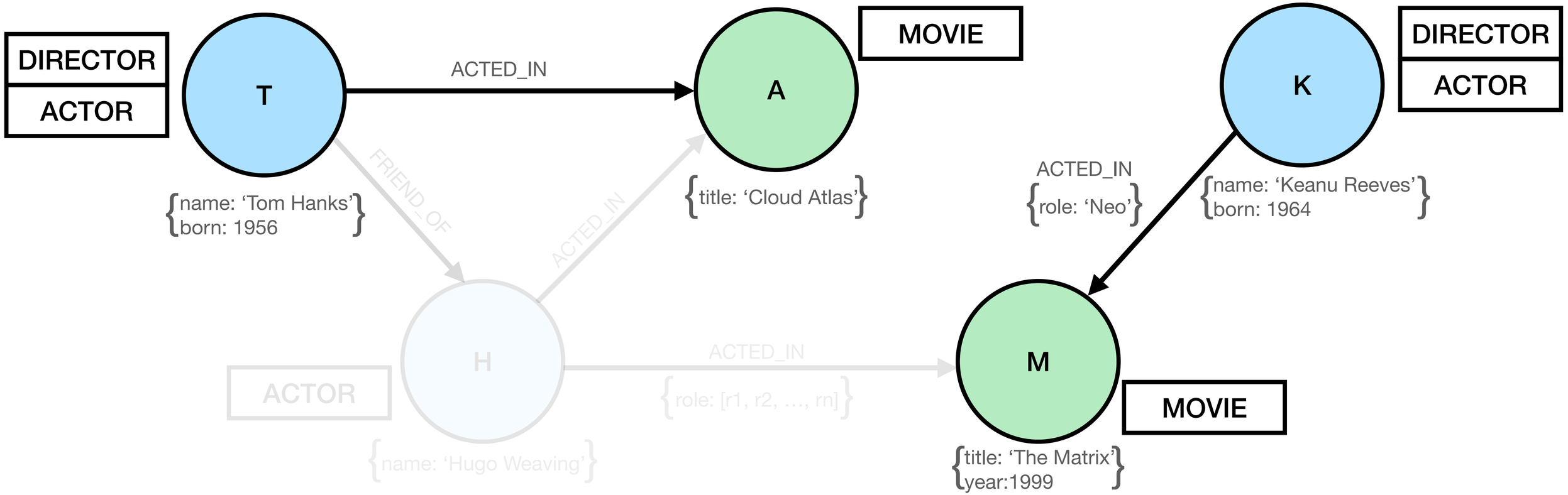
```
(x:Actor {name: ‘Keanu Reeves’})-[:ACTED_IN {role: ‘Neo’}]->(:Movie {title: ‘The Matrix’})
```

Pattern matching example



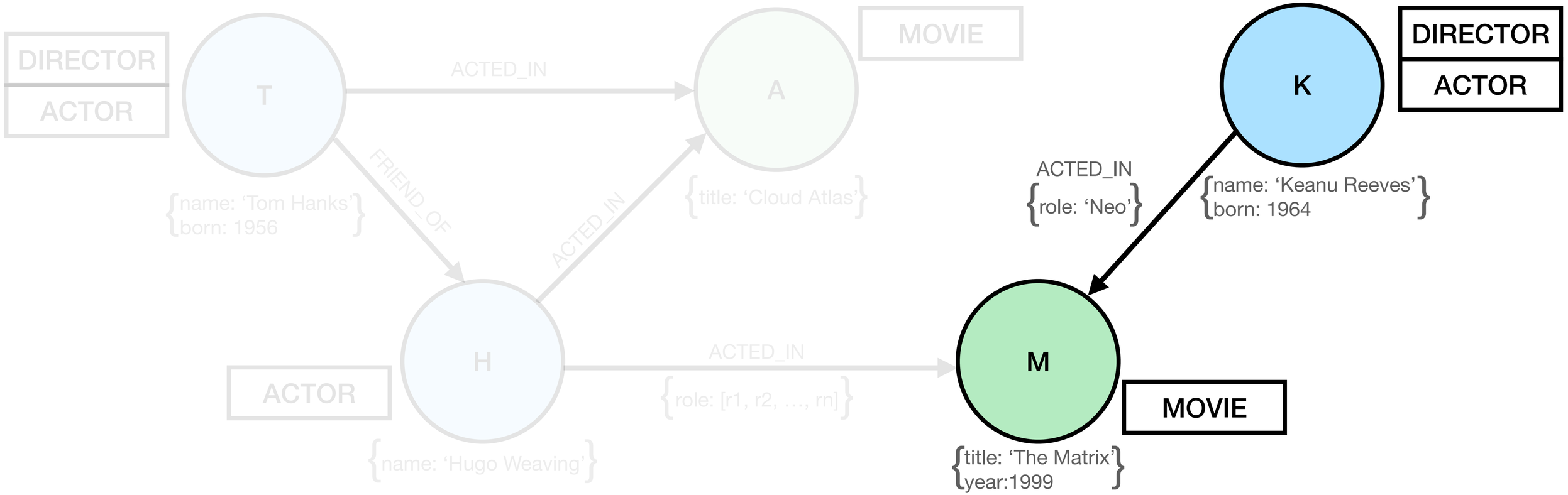
Pattern matching example

Pattern: **(**x:Director**)-[**r:ACTED_IN**]->(**m**)**



Pattern matching example

Pattern: (x:Actor {born:1964})-[r:ACTED_IN]->(m {title:'The Matrix'})



Neo4J’s query language: Cypher clauses

MATCH: precedes a pattern to be matched

WHERE: defines selection conditions to be fulfilled

CREATE: creates nodes and edges according to the provided definition

DELETE: removes nodes and relationships

REMOVE: removes properties and labels

SET: sets and updates values of properties and labels

RETURN: returns either matching subgraphs defined by a query or property values

Cypher - Create

User

UID	Name
A10	Alice
B13	Bob
…	…
Z71	Zach

Product

PID	Desc
P07X	Diapers
P31Y	Beer
…	…
P57Z	Shoes

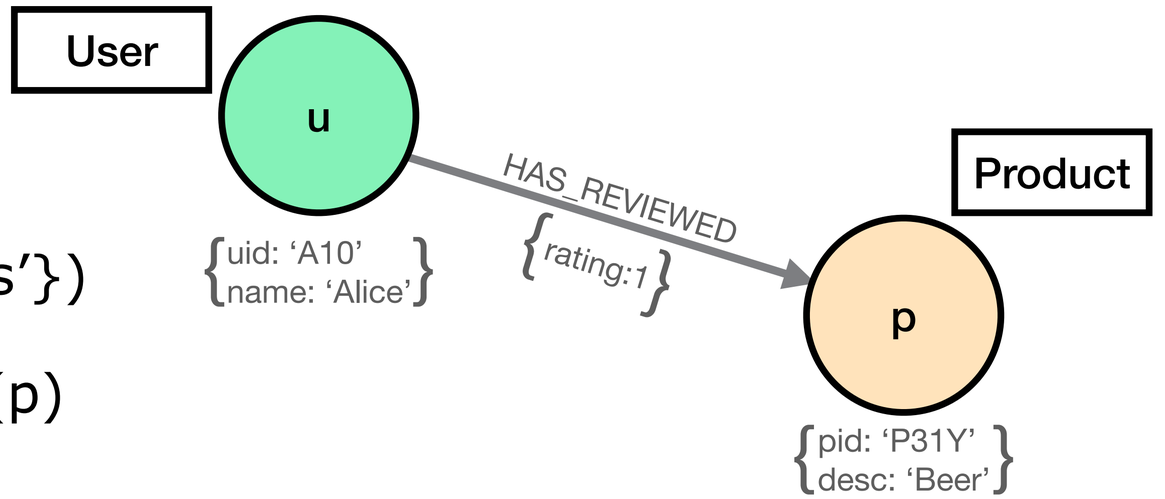
Review

Usr	Prod	Rating
B13	P57Z	4
A10	P31Y	1
…	…	…
B13	P31Y	3

CREATE (u:User {uid: ‘A10’, name: ‘Alice’})

CREATE (p:Product {pid: ‘P31Y’, desc: ‘Diapers’})

CREATE (u)-[:HAS_REVIEWED {rating: 1}]->(p)



Cypher - Read (pattern matching)

```
MATCH (x:User)-[]->(z {desc:'Shoes'})          //return the names of all Users connected to nodes
RETURN x.name                                  //having Shoes as a property

MATCH (a)-[r]-(b)                              //return the names of users who gave hardware
WHERE a.desc = 'Hardware' AND r.rating=5       //products a 5 (NO DIRECTION SPECIFIED)
RETURN b.name

MATCH (a)-[r1]->(c)<-[r2]-(b)                  //return the names of users who gave the same
WHERE r1.rating = r2.rating                    //rating to products they both reviewed
RETURN a.name, b.name, c.desc, r1.rating       //return also the description and review of product

MATCH (a)-[:KNOWS*2..3]->(b)                   //return all pairs of nodes which are at distance
RETURN a, b                                    //at least two and at most 3 through KNOWS edges
```

Cypher - Delete and Update

```
MATCH (x:User)
WHERE x.uid = 'Alice'
DELETE x
```

```
//match all nodes with label User
//select those whose name is Alice
//delete them
```

```
MATCH (x:User)-[r]->(y:Product)
WHERE y.desc = 'Diapers'
DELETE r
```

```
//match all pairs of nodes x and y connected through r
//select only pairs where y.desc='Diapers'
//delete the links between x and y
```

```
MATCH (x:User {uid: 'A10'})-[r]->(y)
SET r.rating = 3
REMOVE y.desc
```

```
//match users with uid='A10' connected to other nodes
//update the rating of r
//remove the property desc of y
```

DEMO

What are we going to do?

- Get familiar with the Neo4J interface
- Manually create a graph database from scratch
- Execute CRUD operations over the graph DB
- Query the database using cypher
- Learn how to load data from external sources